

Limit Order Book Matching Engine

End-to-End Build Plan

Target: Goldman Sachs Engineering Summer Analyst 2027 Application (India)

Owner: Akash | Timeline: 5 weeks | Deadline anchor: applications open July 1st

How to use this document: This file is written as a complete, self-contained spec. Paste any single section (e.g. "Phase 2") into any AI assistant along with the "Core Concepts" section, and ask it to help you implement that phase. No extra context should be needed — the goal, constraints, data structures, and definitions are all included below.

1. Project Overview

A limit order book (LOB) matching engine is the core piece of software that runs every stock exchange and trading venue: it accepts buy/sell orders, matches them against each other using price-time priority, and publishes the resulting trades and updated order book. Goldman Sachs Engineering builds and maintains systems in exactly this space — low-latency, high-throughput, correctness-critical infrastructure — so building a working matching engine directly mirrors the kind of problem their engineers solve daily.

The goal of this project is not just a working demo. It is a **defensible** piece of engineering: correct under concurrent load, benchmarked with real numbers, and documented well enough that you can explain every design decision under interview pressure.

What "done" looks like

- A matching engine that correctly matches limit and market orders using price-time priority.
- A REST endpoint to submit/cancel orders and a WebSocket feed streaming live order book depth.
- Thread-safe handling of concurrent order submissions (single-threaded event-loop model).
- Benchmark numbers: throughput (orders/sec) and p50/p95/p99 matching latency.
- A README with an architecture diagram and a written design-tradeoffs section.
- One resume bullet with real, defensible numbers pulled from your own benchmark run.

2. Tech Stack

Layer	Choice	Why
Core engine language	Java (or Go as alternative)	Performance-oriented, widely used in finance backends; strong typing helps catch errors early.
API layer	REST (order submit/cancel) + WebSocket (live depth/stream)	REST for idempotent actions, WS for continuous market data push.
Concurrency model	Single-threaded event loop per symbol	Avoids lock contention entirely; mirrors how real exchanges isolate matching.
Core data structures	TreeMap (sorted price levels) + ArrayDeque (order queue)	Log (if price level) access, O(1) time-priority ordering within a level.
Load testing	JMH (Java) or wrk/k6 (HTTP layer)	Produces the throughput/latency numbers you need for your resume.
Frontend (optional)	React + native WebSocket client	Live depth-chart visualization; strong for a Superday screen-share demo.
Testing	JUnit 5 (Java) or Go's testing package	Unit tests for matching correctness, partial fills, and priority ordering.
Version control	Git + GitHub, one commit per meaningful change	A clean commit history itself signals engineering discipline to a reviewer.

3. System Architecture

Five components, each with a single responsibility:

- **Order Gateway** — REST endpoint that receives new/cancel order requests, validates them, and hands them to the engine.
- **Matching Engine Core** — one order book instance per symbol; owns all matching logic and state.
- **Event Log** — append-only log of every accepted order and trade, for replay/audit and debugging.

- **Market Data Publisher** — pushes order book depth and trade events to WebSocket subscribers after every match.
- **Benchmark Harness** — a separate load-testing client that fires orders at controlled rates and records latency.

```

Client (REST/WS)
|
v
Order Gateway --validate--> Matching Engine Core (per symbol)
| |
v v
Event Log Market Data Publisher --> WS Subscribers

```

4. Core Concepts (context for you or any AI helping you)

Price-time priority

At each price level, orders are matched in the order they arrived (FIFO). Across price levels, the best price always matches first: highest bid / lowest ask. This is the rule every real exchange uses and the one your engine must implement exactly.

Order types to support

- **Limit order** — buy/sell at a specified price or better; rests on the book if it can't fully match immediately.
- **Market order** — buy/sell immediately at the best available price(s); never rests on the book.
- **Cancel** — removes a resting order by ID.
- **IOC (Immediate-or-Cancel)** — matches whatever it can immediately, cancels the remainder (stretch goal, adds depth for interviews).

Data structures

- Bids: sorted map keyed by price, **descending** (highest price first).
- Asks: sorted map keyed by price, **ascending** (lowest price first).
- Each price level: a FIFO queue of resting orders (preserves time priority).
- Order lookup: a hash map from order ID to its location, for O(1) cancellation.

Matching pseudocode

```

function matchOrder(incomingOrder):
    oppositeBook = asks if incomingOrder.side == BUY else bids
    while incomingOrder.qty > 0 and oppositeBook.hasMatchablePrice(incomingOrder.price):
        bestLevel = oppositeBook.bestPriceLevel()
        restingOrder = bestLevel.peekFront()
        tradeQty = min(incomingOrder.qty, restingOrder.qty)
        recordTrade(incomingOrder, restingOrder, tradeQty)
        incomingOrder.qty -= tradeQty
        restingOrder.qty -= tradeQty
        if restingOrder.qty == 0: bestLevel.popFront()
    if incomingOrder.qty > 0 and incomingOrder.type == LIMIT:
        ownBook.insert(incomingOrder) # rests on the book

```

5. Phase-Wise Build Plan

Phase 1: Core Data Structures & Matching Logic (Week 1)

Goal: A working, network-free matching engine you can drive from unit tests.

- Implement Order, Trade, and PriceLevel classes/structs.
- Implement the order book (sorted bid/ask maps + FIFO queues per level) as described in Section 4.
- Implement addOrder(), cancelOrder(), and the matching algorithm from the pseudocode.
- Write unit tests: exact match, partial fill, multiple fills across price levels, time-priority ordering (two orders at the same price, first one must fill first).

Deliverable: A library (no networking yet) with passing unit tests proving correct price-time priority matching.

Phase 2: Order Types (Week 2)

Goal: Support the full order-type surface a real venue needs.

- Add market order handling (matches immediately, never rests).
- Add IOC or FOK as your advanced order type (pick one, document why).
- Extend cancel to handle already-partially-filled orders correctly.
- Add tests for each new order type, including edge cases (market order with insufficient liquidity).

Deliverable: All order types implemented and unit-tested; this is what proves depth beyond a toy example.

Phase 3: API Layer & Concurrency (Week 3)

Goal: Expose the engine over the network safely under concurrent load.

- Wrap the engine in a REST endpoint: POST /order (submit), DELETE /order/{id} (cancel).
- Add a WebSocket endpoint that streams order book depth after every state change.
- Move to the single-threaded event-loop model per symbol: incoming requests are queued and processed one at a time by a single worker thread per symbol, removing the need for locks inside the matching logic itself.
- Write a concurrency test: fire many simultaneous requests at the same symbol and verify no lost or duplicated matches.

Deliverable: A running service you can hit with curl/Postman and watch the book update live over WebSocket.

Phase 4: Performance & Benchmarking (Week 4)

Goal: Produce the real numbers that go on your resume.

- Build a benchmark harness (or use k6/wrk) that fires orders at increasing rates.
- Measure and record: peak sustained throughput (orders/sec), and p50/p95/p99 matching latency.
- Identify and fix at least one real bottleneck (e.g. object allocation churn, unnecessary copying) — before/after numbers are excellent interview material.
- Log all results into a RESULTS.md with the exact test setup (hardware, order mix, duration).

Deliverable: A results table with real numbers, plus a short note on what you tuned and why it helped.

Phase 5: Frontend, Documentation & Polish (Week 5)

Goal: Make the project easy to demo and easy to defend.

- Build a minimal React page: live bid/ask depth chart driven by the WebSocket feed.
- Write the README: problem statement, architecture diagram, how to run it, benchmark results.
- Write a "Design Tradeoffs" section: why price-time priority, why single-threaded-per-symbol, what you would change to scale across many symbols or add fault tolerance.
- Do a dry run explaining the whole project out loud in under 3 minutes.

Deliverable: A polished GitHub repo: clean README, working demo, benchmark numbers, and a tradeoffs write-up.

6. Extracting Resume-Ready Numbers

Do not put any number on your resume that you cannot reproduce live if asked. Everything below should come directly from your Phase 4 benchmark run, not an estimate.

Numbers to capture

- **Throughput:** peak sustained orders/sec the engine processed without dropping or queuing indefinitely.
- **Latency:** p50, p95, and p99 time from order received to match/rest decision.
- **Correctness under load:** number of orders processed with zero mismatches in your concurrency test.
- **Improvement delta:** if you optimized something in Phase 4, capture the before/after numbers.

Resume bullet templates (fill in your real numbers)

Built a limit order book matching engine in Java implementing price-time priority matching, sustaining [X] orders/sec at p99 latency of [Y] ms under concurrent load.

Designed a single-threaded event-loop architecture per symbol to eliminate lock contention, verified correctness across [Z]+ concurrently submitted orders with zero mismatches.

Reduced matching engine p99 latency from [A] ms to [B] ms by [specific optimization, e.g. eliminating per-order heap allocation].

Interview tip: be ready to explain, without notes, exactly how you measured these numbers and what would change them (different order mix, more symbols, different hardware).

7. How to Use This Plan With Any AI Assistant

This document is intentionally self-contained. To get implementation help for a specific phase from any AI (including a different session of this one), copy in:

- Section 4 (Core Concepts) — so the AI has the exact matching rules and data structures.
- The specific Phase from Section 5 you're working on.
- Your current code, if you have any, and the exact error or question.

Then ask directly, e.g.: *"Using the spec below, help me implement the Order and PriceLevel classes in Java for Phase 1."* No further back-and-forth about project goals should be needed — the constraints are already defined above.

8. Setup Instructions

Repo structure

```
limit-order-book/  
src/  
main/  
OrderBook.java  
Order.java  
PriceLevel.java
```

```
MatchingEngine.java
api/ (REST + WebSocket handlers)
test/
OrderBookTest.java
benchmark/
load_test.js (k6 script) or JMH benchmarks
frontend/
(React app, optional)
README.md
RESULTS.md
```

Steps to initialize

- Create a new GitHub repo: limit-order-book.
- Java: initialize with Maven or Gradle; add JUnit 5 as a test dependency.
- Go alternative: *go mod init github.com/<you>/limit-order-book*.
- Set up a pre-commit hook or CI (GitHub Actions) to run tests on every push — another small signal of engineering discipline.
- Start with Phase 1 only. Do not touch networking code until matching logic is fully unit-tested.

Build plan prepared for Akash's Goldman Sachs Engineering Summer Analyst 2027 application prep.